
DYNAMIC WEIGHT GRAFTING: LOCALIZING FINE-TUNED FACTUAL KNOWLEDGE IN TRANSFORMERS

Anonymous authors
Anonymous submission

ABSTRACT

Finetuning can teach an LLM a new fact, but it remains unclear where that fact is used during generation: does the model enrich entity representations as soon as they appear, recall relation information only at the final prediction position, or rely on multiple redundant routes? This paper makes three contributions toward answering that localization question. First, we introduce **dynamic weight grafting**, an analysis technique that selectively grafts *subsets of weights* from a finetuned model onto a pretrained model at chosen token positions and components, avoiding the residual-stream overwrite behavior of standard activation patching. Second, using this technique, we identify two retrieval pathways for finetuned relation information: an “enrichment” pathway that adds relation information while processing entity tokens (e.g., “Zendaya” in “Zendaya co-starred with Timothée Chalamet”) and a “recall” pathway that retrieves the relation at the final token position before generating the target fact. Third, we localize the “recall” pathway to concrete components: task-specific attention mechanisms and an entity-specific extraction step in the feedforward networks of the final layers before prediction. Across models, the enrichment and recall pathways can be individually sufficient in some cases, jointly necessary in others, and together nearly recover finetuned behavior. By intervening on parameters and components rather than only on activations, dynamic weight grafting makes the mechanisms of finetuned knowledge retrieval more directly observable.

1 INTRODUCTION

When a new pope is elected, a finetuned LLM may learn to answer “Who’s the pope?” correctly, but the mechanism behind that update is not obvious. LLMs store and retrieve many relationships from pretraining (Petrone et al., 2019; Roberts et al., 2020); our question is how a model uses its parameters after finetuning adds new factual relationships. In reviewer-facing terms, the paper studies not only whether a finetuned model can state a new fact, but where and by which internal mechanisms that fact is retrieved during generation.

We focus on a concrete localization problem. After finetuning on a new relation, the model might store the update mainly with the entity itself, enrich the entity representation in lower or middle layers, or wait until higher layers near the next-token prediction to recall the relation in response to the prompt. A useful contribution must therefore be more specific than “the model knows the fact”: it should identify the token positions and model components that are sufficient, necessary, or specialized for retrieval.

The first contribution is a tool for isolating *mechanisms* rather than only states. For example, if a final-layer feedforward network appears important for answering a newly updated pope question, we want to know whether that component actively recalls the new relation or merely receives information already carried forward by attention. Existing localization methods such as *activation patching* (Meng et al., 2022a) and ablations (Geva et al., 2023) identify components that affect next-token prediction, but they do so by modifying or replacing activations. That intervention can erase upstream computations, including earlier “enrichment” of an entity with factual information (Geva et al., 2023). Consequently, activation-level interventions can confound active extraction with passive transmission, making it difficult to establish both necessity and sufficiency for a retrieval mechanism.

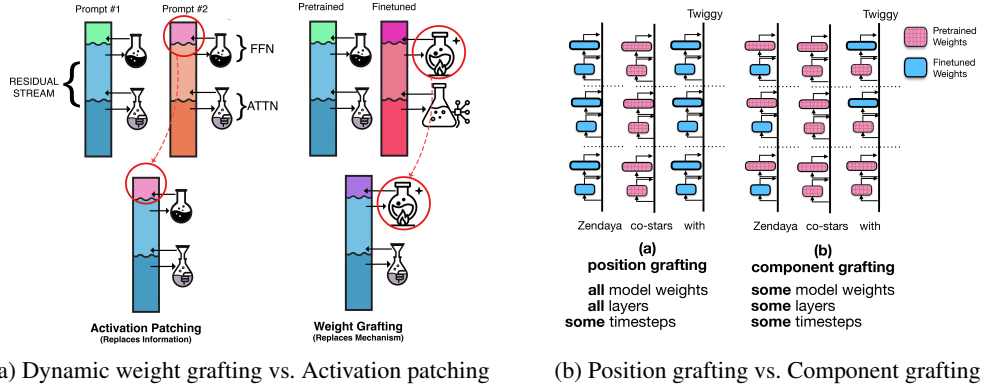


Figure 1: Dynamic weight grafting is the paper’s methodological contribution for analyzing *model mechanisms* in finetuned knowledge retrieval. Instead of replacing activations, we swap *small subsets* of a pretrained model’s weights with weights from a supervised-finetuned (SFT) model. (a) **Dynamic Weight Grafting vs. Activation Patching:** activation patching replaces activations from another run, whereas dynamic weight grafting replaces specific parameter matrices and therefore tests mechanisms while preserving upstream computation. (b) The two grafting schemes used throughout the experiments: position grafting swaps the full model at selected token positions, and component grafting dynamically blends pretrained and finetuned components at selected positions.

To address this gap, we propose **dynamic weight grafting**,¹ a mechanism-level intervention that swaps selected weights from a finetuned model into a pretrained model during generation. Because the swap can be restricted to particular layers, components, and token positions (see Figure 1b), the method asks reviewer-relevant localization questions directly: which finetuned mechanisms are sufficient to reproduce the finetuned output, and which positions are necessary when all other finetuned mechanisms are available? Dynamic weight grafting combines the upstream-computation preservation of model grafting (Panigrahi et al., 2023; Ilharco et al., 2022) with the targeted causal logic of activation patching (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a).²

The second contribution is an empirical decomposition of finetuned relation retrieval into two pathways. In position-level grafting, grafting only the first entity or only the last token is sometimes enough to **reproduce the finetuned model’s** relation-completion behavior. This supports an “enrichment” pathway, where relation information is added when the entity is processed and later carried forward (Meng et al., 2022a; Geva et al., 2023; 2022), and a distinct “recall” pathway, where later finetuned mechanisms extract a relation completion from a representation that was not itself enriched by finetuning. The two positions together nearly recover full finetuning performance, while grafting everything *except* them gives near-zero relation-completion accuracy. Thus the contribution is not just a visualization of where information appears, but a necessary-and-sufficient localization of the two retrieval routes available in this setting.

2 METHODOLOGY

The methodological goal is to turn a finetuned-vs.-pretrained comparison, with parameters θ^{pre} and θ^{ft} , into a structured localization test. We want to distinguish gradual token-by-token construction from final-position “look back” recall, while keeping the residual stream available to downstream computations. Activation patching intervenes on the *information* flowing through the model by replacing residual-stream vectors or component inputs/outputs. Dynamic weight grafting instead intervenes on model *mechanisms*: it keeps the computation stream in place while changing which parameterized operation is applied at selected positions. This section frames the contribution by reviewing relation completion, activation patching, and prior weight grafting before defining the

¹Code will be released upon acceptance.

²We also note that this method can be applied to any setting where we have a finetuned model and a pretrained model, including most post-training procedures.

dynamic grafting procedure used to localize finetuned knowledge retrieval. See Figure 1 for the activation-patching comparison.

Relation Completion We use relation completion as the controlled behavior whose mechanisms we localize. Classic relation extraction identifies the semantic relation (r) between a subject (s) and an object (o), yielding an (s, r, o) tuple (Zhao et al., 2024; Hinton et al., 1986). Our generative version fixes the subject and relation in a natural-language prompt and asks which mechanisms make the model generate the object. This task supplies the paper’s main experimental handle: successful completion can be compared across pretrained, finetuned, and grafted models. See Figure 1b for an example.

Activation Patching Prior localization work often tracks information flow through residual-stream vectors (hidden states) $\lambda(t, l)$ at token t and layer l . We use *activation patching* to refer broadly to methods that replace some vectors $\lambda(t, l)$ with new vectors $\tilde{\lambda}(t, l)$, whether the intervention swaps the full residual stream, a subspace, or the input/output of a specific model component (Heimersheim and Nanda, 2024; Goldowsky-Dill et al., 2023; Meng et al., 2022a). This family of methods provides an important baseline for causal localization, but its intervention target is the activation, not the parameterized mechanism.

The limitation motivating our method is that activation patching *overwrites* information that may be part of the mechanism under study. If we patch a pretrained residual stream into a finetuned model at an early final-token layer, we may erase information used to select earlier positions through attention; if we patch later, we may erase information that attention has already imported. Even the most natural component-localization patch—inserting a pretrained residual stream into the finetuned computation graph—leaves two confounds: subsequent computations still use finetuned parameters, and earlier residual streams may already encode the relevant relation. We therefore graft *subsets of* finetuned parameters, so that the ordinary computation can continue while the tested mechanism changes.

Weight Grafting Our method builds on static weight grafting, which asks whether a sparse subset of finetuned parameters can recover finetuned behavior. In previous work, Panigrahi et al. (2023) define a *grafted* model $\tilde{\theta}$ using a mask γ ; they learn a sparse mask over model parameters to recover finetuned performance:

$$\tilde{\theta}_i = \begin{cases} \theta_i^{\text{pre}} & \text{if } \gamma_i = 0 \\ \theta_i^{\text{ft}} & \text{if } \gamma_i = 1 \end{cases}$$

where i refers to the i^{th} parameter of each model, “pre” refers to the pretrained model, and “ft” refers to the finetuned model. Equivalently, Ilharco et al. (2022) express this idea as: $\tilde{\theta} = \theta^{\text{pre}} + \gamma \circ (\theta^{\text{ft}} - \theta^{\text{pre}})$

This static setup establishes that finetuned weights can be combined with pretrained weights, but it does not by itself reveal where in the sequence the finetuned mechanism is used. Our contribution is to make grafting dynamic: the chosen components can change at each token position (see Figure 1b). This lets us localize behavior jointly to model components and token positions while preserving information flow over the full sequence, a perspective emphasized in previous work (Geva et al., 2022; 2023; Ferrando and Voita, 2024).

Dynamic Weight Grafting Dynamic weight grafting turns mechanism localization into an explicit sufficiency test. We search for a **subset** of weights and token positions that can **replicate full finetuned outputs** when inserted into the pretrained model. Each Transformer component at each token position is treated as an intervention target: for example, we can replace only the feedforward networks at the first entity token positions with their finetuned counterparts and ask whether that mechanism is enough to recover relation completion.

Formally, the method compares two models θ^A and θ^B with ordered weight matrices $\theta^A = [\theta_1^A \cdots \theta_M^A]$ and $\theta^B = [\theta_1^B \cdots \theta_M^B]$. Let γ be a $1 \times M$ mask over components, where each θ_c^A denotes a specific component such as the W^Q matrix at layer 12. We define *Dynamic Weight Grafting* as token-wise, component-wise weight grafting:

$$\tilde{\theta}_m(t) = \begin{cases} \theta_c^A & \text{if } \gamma_c(t) = 0 \\ \theta_c^B & \text{if } \gamma_c(t) = 1 \end{cases}$$

where c indexes the model component and t indexes the token position. Thus, while the residual stream for a token is processed, the grafting configuration selects whether each component comes from model A or model B. This is the mechanism-level intervention shown in Figure 1b: for instance, we can use finetuned feedforward networks only in the second half of the model at the last token to test whether that final-position mechanism is sufficient for recall. In our experiments, θ_i^A is the pretrained model θ_i^{PRE} and θ_i^B is the finetuned model θ_i^{SFT} .

3 EXPERIMENTS & RESULTS

Models We evaluate the localization claims on four pretrained Transformer-based decoder-only language models: Llama3 Grattafiori et al. (2024), Pythia 2.8b Biderman et al. (2023), GPT2-XL Radford et al. (2019), and Gemma 1.1 Gemma Team et al. (2024). The models are similar enough in scale to compare grafting behavior, but they differ architecturally in ways that let us see whether the same contribution-level pattern holds across model families. See Table 2 in Section A.3.1 for details.

Data The experimental contribution requires controlled facts, so we follow Allen-Zhu and Li (2023) and finetune on templated supervised data where the exposed relations are known. Each dataset contains 1,000 synthetic metadata tuples and approximately 10,000 finetuning examples created from five article-style templates and five QA templates. We use three dataset variants to separate dependence on known entities from dependence on the relation update itself: (1) **Fake Movies, Real Actors**, with real actor names and programmatically generated movie titles; (2) **Fake Movies, Fake Actors**, with programmatically generated movie and actor names; and (3) **Real Movies, Real Actors (Shuffled)**, with real movies and actors but shuffled relations (e.g., “Keanu Reeves stars in The Departed alongside Meryl Streep”). The main text reports Fake Movies, Real Actors results, while Section C shows that the qualitative pattern is similar across datasets. We also test Gemma on Wikipedia articles for movies released after the model release date to check whether the mechanism pattern extends beyond templated data. See Section A.2.3, Section A.2.4, Section A.2.2, and Section A.3.3 for templates, examples, dataset construction, and training details.

3.1 WHICH POSITIONS ARE SUFFICIENT FOR RELATION COMPLETION?

We first make the pathway contribution visible with position grafting: during generation, we graft all model weights at selected token positions and leave all other positions pretrained (see Figure 1b). This all-or-nothing position intervention asks which token positions are sufficient for recovering the finetuned relation-completion behavior. When a position is grafted, its keys and values are also computed with the grafted model, even if later positions use pretrained parameters, so the chosen positions are fully processed by the finetuned model. The main result is that grafting *both* the first entity tokens and the final token before prediction nearly recovers full finetuning performance across tested models (Figure 2). See §3.1 for the top-5 accuracy choice.

The same experiments separate two interpretable routes. First, grafting only the first entity tokens can recover nontrivial top-5 performance, consistent with an enrichment pathway in which entity representations acquire factual information early and carry it forward (Meng et al., 2022a; Geva et al., 2023; 2022). Second, grafting only the final token can also be sufficient in some cases, revealing a recall pathway that extracts relation information without finetuned subject enrichment. This directly supports the paper’s central empirical contribution: finetuned relation completion is not localized to a single universal site, but decomposes into at least these two recoverable pathways.

The individual pathways are weaker than their combination, but they are still meaningful sufficiency witnesses. For several models and sentence templates, either enrichment alone or recall alone produces good relation-completion performance. In Gemma-1.1, the “recall” pathway alone achieves 53% top-5 accuracy, illustrating that final-token mechanisms can recover a substantial fraction of the finetuned behavior even without first-entity grafting.

A Note on Top-5 Accuracy For Figure 2, we report top-5 accuracy for the correct relationship token: an example is correct when the desired relationship token appears among the five highest-probability next-token choices. This metric makes the localization results easier to interpret in our open-ended generative setting. Models sometimes assign high probability to context entities or common tokens such as “the,” and actor names can have several plausible tokenizations (e.g., “R”, “

Rob", "Rob", " Robert"). Top-5 accuracy therefore captures whether a grafted pathway raises the correct relation completion into the model’s plausible outputs, while remaining a lower bound on retrievable relation knowledge (Jiang et al., 2020). Varying k changes the scale but not the qualitative pattern; see Section C.2.

The metric also exposes a limitation of grafted hybrids that matters for interpreting the contribution. A model may combine features that support the relation with features that do not, causing probability mass to move toward a prompt-unconditional prior and high-frequency tokens. To make this failure mode visible rather than hiding it behind accuracy alone, Section C.1.2 reports token-rank results. Understanding when grafted models default to the unconditional prior remains an open direction.

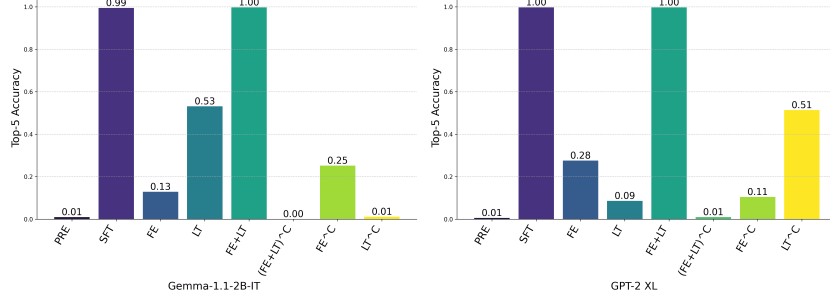


Figure 2: **Position-level contribution:** top-5 accuracy on the headline test sentence shows that the first entity (FE) and last token (LT) are the salient positions for recovering finetuned relation completion. Grafting configurations are PRE (pretrained baseline), SFT (supervised finetuning baseline), FE, LT, FE+LT, (FE+LT)^C (everything except FE and LT), FE^C, and LT^C. Grafting FE+LT nearly recovers SFT performance, while grafting everything *except* FE+LT stays near pretrained performance. We show Gemma and GPT-2 XL here; Llama and Pythia results are in Section C.

3.1.1 NECESSITY FOR RELATION COMPLETION

Sufficiency alone would not rule out other retrieval routes, so we next test necessity. We graft the complement of the first entity and last token, (FE+LT)^C, meaning *all token positions* except the first entity tokens and the final token. If other positions contained an independent recovery pathway, this complement should retain finetuned performance. Instead, top-k accuracy is near zero for all models and comparable to the pretrained baseline (Figure 2), making FE and LT jointly necessary in this setup.

One model-specific caveat is that GPT2-XL performs better on LT^C than the other models, whose accuracy under this grafting scheme is near zero. We therefore also test prompts that include the movie title (Section C.5). The movie title alone does not recover finetuned performance, but combining the movie title with the last token improves over the last token alone for all models; combining the movie title with the first entity is inconsistent across models. We return to this architecture-sensitive variation in §4.

3.2 IS IT THE POSITION OR THE TOKEN THAT MATTERS?

The headline template leaves one ambiguity for the position-level claim: the last token before generation is also the preposition associated with the relation (e.g., stars **in**). To distinguish a last-position effect from a relation-token effect, we constructed the QA test in Table 1, where the final token before generation is neither the relation nor a relation-associated preposition. The same qualitative pattern appears: **the last token remains responsible for relation knowledge retrieval even when it is not itself a relation or preposition token**. This supports the contribution that the recall pathway is tied to final-position computation, not merely to relation-token identity; see Table 1 for the contrasting templates and Section C.1.1 for the results.

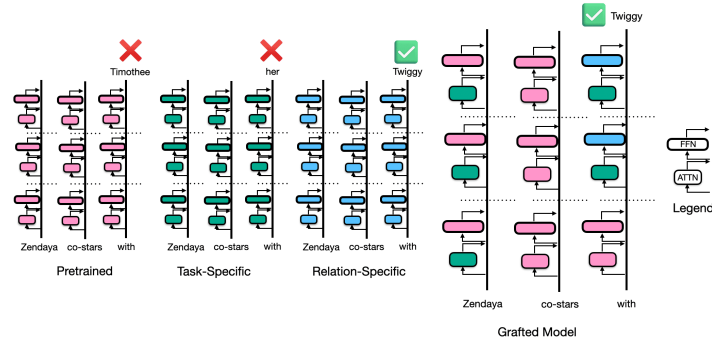


Figure 3: Component-level contribution for the “recall” pathway. The **task-specific** model has seen data with the same form as the test task but not the test relation, while the **relation-specific** model has seen the test relations. Hybrid grafting shows that recall depends on task-specific attention at the first entity and final token, together with **relation-specific** extraction mechanisms in final-layer feedforward networks before next-token prediction.

3.3 CAN WE LOCALIZE THE “RECALL” PATHWAY TO INDIVIDUAL MODEL COMPONENTS?

Having established the two salient positions, we next make the component-level contribution explicit. Prior work shows that next-token prediction blends information propagation through attention heads with token promotion through feedforward networks (Geva et al., 2020; 2022; 2023). We therefore ask whether final-token “recall” is implemented mainly by attention, feedforward networks, or their interaction: **which components learn and execute the new relation?**

Component grafting Component grafting refines position grafting by swapping individual Transformer-block components at a chosen token position (Figure 1b). This lets us test whether a recall effect is carried by attention projections, the output projection, the feedforward network, or a combination. We use the following block schematic, abstracting over model-specific normalization and residual-placement details.³:

$$\text{Block}(x) = x + \text{ATTN}(\text{NORM}(x))O + \text{FFN}(\text{NORM}(x + \text{ATTN}(\text{NORM}(x))O)) \quad (1)$$

Here NORM is either LayerNorm or RMSNorm, ATTN is attention, FFN is the feedforward network, and O is the multi-head attention output projection. We graft weight matrices such as W^K , W^Q , and W^V at selected positions to isolate their contribution. The component-grafting results support two claims about recall: (1) relation retrieval is localized to the output projection matrix and feedforward networks at the final token position, and (2) the pathway also requires *task-specific* attention on the first entity. Figure 3 summarizes this component-level localization.

Training on disjoint data to localize relation completion To separate task-format mechanisms from relation-specific mechanisms, we train two finetuned models. The *relation model* is trained on the full dataset, as in the position-grafting experiments. The *task model* is trained on text with the same semantic and syntactic form but without the specific relation information tested at evaluation. This design lets a hybrid graft ask whether a component contributes generic task machinery or relation-specific recall. See Section A.2.5 for task-model and relation-model examples.

The split relies on **the reversal curse**: Berglund et al. (2023) and Allen-Zhu and Li (2023) show that training on one direction of a relationship (“Werner Herzog starred in a movie with Nicolas Cage”) does not reliably teach the reverse direction (“Nicolas Cage starred in a movie with Werner Herzog”) (bad, 2009). Our “task-specific” models see only one direction of each relationship, with the

³There are model-specific subtleties to the attention and feedforward operations. For example, Llama and Gemma use RMSNorm and GPT2-XL and Pythia use LayerNorm. Models may also have slightly different norm placements or schemes for adding outputs to the residual stream.

same entity order across paraphrases, while our “relation-specific” models see *both* directions. The resulting contrast gives component grafting a sharper interpretive target: task competence without the evaluated relation versus relation knowledge itself.

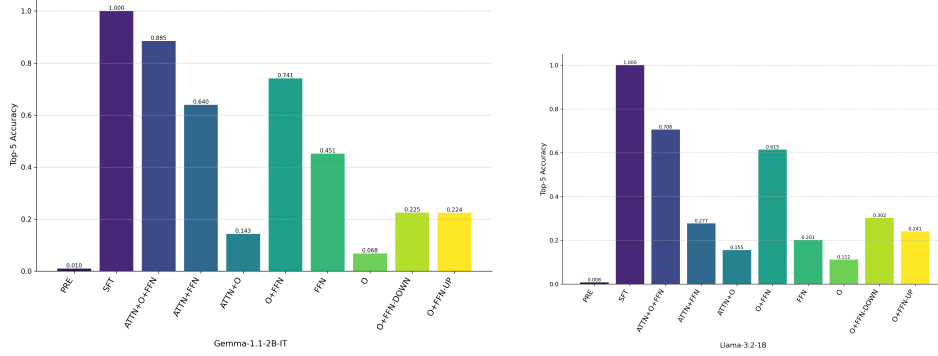


Figure 4: **Component-level localization of recall:** top-5 accuracy on the headline test sentence after grafting last-token weights from models finetuned on both directions of a symmetric relationship. For Gemma and Llama, the output projection matrix and feedforward networks in the final quarter of layers recover most of the finetuned performance, identifying these components as the main relation-specific recall mechanism.

Grafting with a hybrid model We first graft from the *relation* model into the *task* model at the final token position to identify the relation-specific portion of recall. Results for Gemma and Llama3 appear in Figure 4. As shown in Figure 4, this experiment isolates the final-token components that drive recall.⁴ Grafting the O matrix and full FFN nearly matches grafting the *full* attention mechanism plus the FFN, indicating that finetuning learns operations in the O matrix that enable final-layer FFNs to trigger the correct recall mechanism before predicting the entity. The remaining attention components contribute little when both source and target models have learned the same task form. The O matrix is important: removing it and grafting only the FFN reduces top-5 accuracy by 29%.

We then construct a three-model hybrid from (1) a pretrained model with none of the dataset relations, (2) a *task* model, and (3) a *relation* model. Guided by the previous result, we graft *task* ATTN and *relation* O+FFN at the last token, then vary which *task* components are grafted at the first entity. In Figure 5, grafting *task* ATTN at the first entity and, at the last token, *task* ATTN plus *relation* O+FFN recovers 63% top-5 accuracy, showing that recall requires both task-specific attention routing and relation-specific final-token extraction.

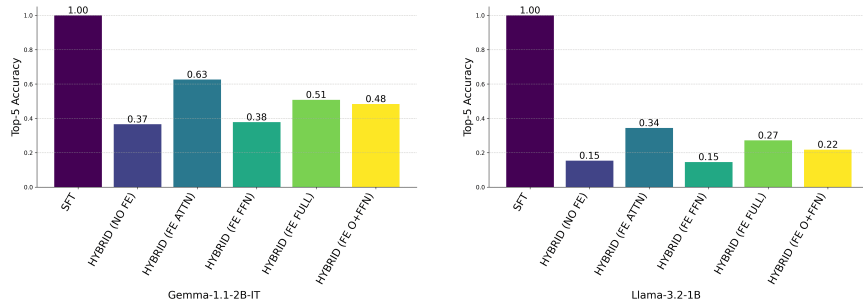


Figure 5: **Hybrid component-grafting result:** top-5 accuracy on the headline test sentence when combining a *task* model and a *relation* model on a pretrained base. The last token always receives *task* ATTN and *relation* O & FFN in the final half of layers; the first entity (FE) receives different *task* components across conditions. The pattern isolates task-specific first-entity attention as a required support mechanism for relation-specific final-token recall.

⁴GPT2-XL and Pythia had much weaker “recall” results than Gemma and Llama—we present results for GPT2-XL in Section C.9.

3.4 DO THESE MECHANISMS APPLY TO NON-TEMPLATED DATA?

We next test whether the same contribution-level pattern appears outside templated data. We finetune Gemma on twenty Wikipedia articles, plus five LLM-generated rephrases per article, about movies released after the model’s release date. This setting is less controlled: models may be sensitive to article structure and entity order, and some film information could have appeared in training despite the release-date filter. Even with these caveats, the first entity plus last token nearly recover full finetuning performance, the individual enrichment and recall pathways recover some performance, and the complement of first entity plus last token matches the pretrained baseline. Because the individual pathways are weaker than in the synthetic setting, we report both top-5 and top-50 accuracy.

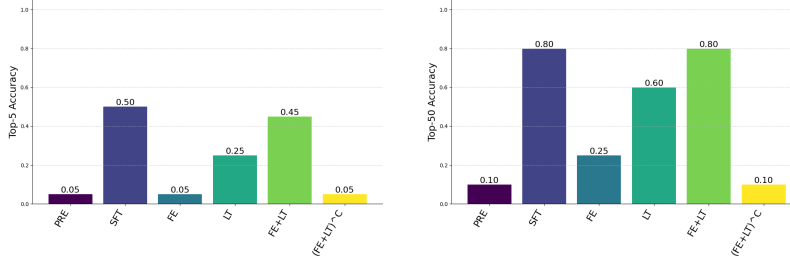


Figure 6: **Non-templated-data check:** accuracy for Gemma finetuned on Wikipedia articles about movies released after the model’s release date. We report both top-5 and top-50 accuracy because the individual enrichment and recall pathways are weaker than in the synthetic experiments. The first entity plus last token still nearly recover finetuning performance, while the complement of those positions remains at pretrained-model performance.

4 DISCUSSION

Dynamic weight grafting supports three paper-level contributions:

- **Pathway localization:** after supervised finetuning on new relation information, relation completion is concentrated at the entity tokens and final token position; either pathway can be sufficient, and together they nearly recover full finetuning performance.
- **Necessity evidence:** at least one of these pathways is required in our setting, because grafting everything *except* the entity tokens and final token position results in near-zero top-5 accuracy.
- **Component localization:** component grafting (see Figure 1b) localizes final-token “recall” to the output projection matrix and feedforward networks, supported by task-specific attention at the first entity and final token.

The pathway contribution is consistent across the main experiments, but its strength is architecture-dependent. The last-token “recall” pathway is much stronger in the tested Gemma and Llama3 models than in GPT2-XL and Pythia. These models differ in normalization, positional embeddings, activation functions, tied versus untied embeddings, attention mechanisms, training data, and training dynamics; Table 2 summarizes these differences. We hypothesize that the more recent architectures provide attention mechanisms that support stronger recovery of relation information, but we do not identify a single causal architectural factor.

A second implication is that the same localization pattern appears for known entities, unknown entities, and shuffled real-entity relations. Results are similar for Fake Movies, Real Actors; Fake Movies, Fake Actors; and Real Movies, Real Actors, even though the last case *overwrites* existing associations. This suggests that narrow relation information can be manipulated by finetuning within the tested context for both known and unknown entities. The claim is intentionally limited: prior knowledge-editing work shows that such updates often fail to generalize across contexts (Zhong et al., 2023b; Hase et al., 2021).

The method contribution also changes the interpretation of component roles. Because weight grafting replaces mechanisms without deleting the computations already present in the residual stream (see Section 2), it can produce different localizations from destructive interventions. Our component-grafting results in 3.3 differ in emphasis from Geva et al. (2023), who show that knocking out attention harms relation completion more than knocking out feedforward networks. In our setting, once a model has learned the task form, O matrices and final-token feedforward networks are nearly sufficient to recover relation completion as long as task-specific attention is available. This is the methodological payoff: mechanism replacement can separate task routing from relation extraction more finely than interventions that erase or replace activations.

Future Work The clearest next step is to explain the alternative enrichment pathway that appears in Pythia and GPT2-XL (see 3.1.1). We hypothesize that some pre-final positions can extract relation information from an enriched entity and pass it to the final token for processing; GPT2-XL appears to have the strongest enrichment pathway, possibly because it has more layers than the other tested models. A second extension is multi-hop retrieval, since all experiments here are single-hop. Finally, although we localize relation completion to components in some settings, we do not interpret the internal parameters of those components. Parameter-space interpretability methods (Ilharco et al., 2022; Jain et al., 2024; Millidge and Black, 2022) may be useful for analyzing the specific O matrices and FFNs identified by dynamic weight grafting.

Limitations The contribution should be read with several limits. First, most experiments use a synthetic knowledge-retrieval task, so the results may not fully generalize to more complex sentences or more varied finetuning data. Second, we measure retrieval success through top-k accuracy and correct-token rank during next-token prediction; the method does not capture forms of knowledge that do not affect the next-token distribution. Third, the space of possible grafting schemes is combinatorial, and we evaluate only a targeted subset designed around first-entity and final-token hypotheses. Fourth, features may interact or cancel in ways that hide additional retrieval routes. Fifth, dynamic weight grafting can push probability mass toward common tokens such as “the” or punctuation, plausibly because hybrid feature blends make the model uncertain and closer to a prior distribution. Finally, compute limits restrict us to smaller models; larger models may implement finetuned relation retrieval differently.

5 RELATED WORK

Mechanistic Interpretability & Knowledge Editing Our contribution sits between mechanistic interpretability and knowledge editing. Interpretability work intervenes on Transformer language models to explain behavior (Vig et al., 2020; Geiger et al., 2021), including work on subject-object relations (Meng et al., 2022a; Geva et al., 2023; Hernandez et al., 2023; Yu et al., 2023). Follow-up work by Hase et al. (2023) and Wang and Veitch (2025) cautions that successful editing is not necessarily evidence of localization, because Transformer models can often be edited in multiple places to change stored knowledge. We address a different but related question: after ordinary finetuning on new relation information, which mechanisms are actually used during retrieval? Other relevant work studies information flow with gradient-based methods (Ferrando and Voita, 2024; Kramár et al., 2024; Kobayashi et al., 2023), identifies circuits for specific tasks (Wang et al., 2022; Nanda et al., 2023; Zhong et al., 2023a; Hanna et al., 2023; Merullo et al., 2024), and relates dictionary-learning features to next-token prediction (Cunningham et al., 2023; Bricken et al., 2023; Gao et al., 2024; Ameisen et al., 2025). Work comparing representations across models (Huh et al., 2024; Kornblith et al., 2019; Raghu et al., 2021; Wolfram and Schein, 2025) and studying knowledge extraction during generation (Balesni et al., 2024) further motivates mechanism-level localization, complementing prior entity-enrichment analyses (Geva et al., 2023). The reversal-curse findings of Berglund et al. (2023) and Allen-Zhu and Li (2023) are especially important for our component experiments, because they show that training on “A is B” need not teach “B is A.” Finally, knowledge-editing methods directly modify weights (Meng et al., 2022a;b; De Cao et al., 2021; Hase et al., 2021; Mitchell et al., 2021; Zhu et al., 2020), multi-hop editing benchmarks test downstream consistency (Zhong et al., 2023b; Cohen et al., 2023), and attention-head analyses study task performance and retrieval (Sakarvadia et al., 2023; Yin et al., 2024); dynamic weight grafting complements these lines by replacing mechanisms during generation rather than editing for deployment.

Interpreting Model Parameters Prior parameter-space work localizes directions or regions in model weights (Millidge and Black, 2022), often to support transfer learning or model merging (Ilharco et al., 2022; Yadav et al., 2023). Closest to our method, Panigrahi et al. (2023) use weight grafting for encoder-only language models and find sparse weight selections that transfer natural-language-understanding performance. Gueta et al. (2023) identify a finetuned-model “knowledge region” that supports finetuned task performance. Our contribution differs by making grafting token- and component-conditional during generation, so the same parameter-space idea can localize retrieval pathways rather than only recover task accuracy.

6 CONCLUSION

This paper contributes a mechanism-level method and a corresponding localization result for finetuned factual knowledge. Methodologically, dynamic weight grafting swaps *targeted subsets* of finetuned parameters into a pretrained model at selected token positions and components, preserving upstream computation while testing sufficiency and necessity. Empirically, the method reveals two single-hop relation-retrieval pathways: “enrichment” at the first entity and “recall” at the final token position. Component grafting further localizes recall to task-specific attention at the first entity and final token, plus relation-specific extraction through the O matrix and final-layer feedforward networks. The overall contribution is a clearer parameter- and component-level account of how finetuned relation information is retrieved during generation, with explicit limits around task scope, model scale, and unexplored grafting schemes.

7 REPRODUCIBILITY STATEMENT AND LLM USAGE

Code will be released upon acceptance. To make the methodological contribution reproducible, Section A.1 provides pseudocode for dynamic weight grafting, Section A.2 documents dataset creation and examples, and Section A.3 gives model and finetuning details including hyperparameters. LLMs were used during manuscript preparation for rephrasing, fluency improvements, identifying confusing passages, and LaTeX troubleshooting. LLMs were also used during experimentation for autocomplete and debugging assistance, and some data-extraction steps were generated with LLM agents.

REFERENCES

- Bad lieutenant: Port of call new orleans, 2009. Film.
- Z. Allen-Zhu and Y. Li. Physics of language models: Part 3.1, knowledge storage and extraction. *arXiv preprint arXiv:2309.14316*, 2023.
- E. Ameisen, J. Lindsey, A. Pearce, W. Gurnee, N. L. Turner, B. Chen, C. Citro, D. Abrahams, S. Carter, B. Hosmer, J. Marcus, M. Sklar, A. Templeton, T. Bricken, C. McDougall, H. Cunningham, T. Henighan, A. Jermyn, A. Jones, A. Persic, Z. Qi, T. Ben Thompson, S. Zimmerman, K. Rivoire, T. Conerly, C. Olah, and J. Batson. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- M. Balesni, T. Korbak, and O. Evans. The two-hop curse: Llms trained on $a \rightarrow b$, $b \rightarrow c$ fail to learn $a \rightarrow c$. *arXiv preprint arXiv:2411.16353*, 2024.
- L. Berglund, M. Tong, M. Kaufmann, M. Balesni, A. C. Stickland, T. Korbak, and O. Evans. The reversal curse: Llms trained on "a is b" fail to learn "b is a". *arXiv preprint arXiv:2309.12288*, 2023.
- S. Biderman, H. Schoelkopf, Q. G. Anthony, H. Bradley, K. O’Brien, E. Hallahan, M. A. Khan, S. Purohit, U. S. Prashanth, E. Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pages 2397–2430. PMLR, 2023.
- T. Bricken, A. Templeton, J. Batson, B. Chen, A. Jermyn, T. Conerly, N. Turner, C. Anil, C. Denison, A. Askell, R. Lasenby, Y. Wu, S. Kravec, N. Schiefer, T. Maxwell, N. Joseph, Z. Hatfield-Dodds, A. Tamkin, K. Nguyen, B. McLean, J. E. Burke, T. Hume, S. Carter, T. Henighan, and C. Olah. Towards monosemanticity: Decomposing language models with dictionary learning. *Transformer Circuits Thread*, 2023. <https://transformer-circuits.pub/2023/monosemantic-features/index.html>.
- R. Cohen, E. Biran, O. Yoran, A. Globerson, and M. Geva. Evaluating the ripple effects of knowledge editing in language models, 2023. URL <https://arxiv.org/abs/2307.12976>.
- H. Cunningham, A. Ewart, L. Riggs, R. Huben, and L. Sharkey. Sparse autoencoders find highly interpretable features in language models, 2023. URL <https://arxiv.org/abs/2309.08600>.
- N. De Cao, W. Aziz, and I. Titov. Editing factual knowledge in language models. *arXiv preprint arXiv:2104.08164*, 2021.
- J. Ferrando and E. Voita. Information flow routes: Automatically interpreting language models at scale. *arXiv preprint arXiv:2403.00824*, 2024.
- L. Gao, T. D. la Tour, H. Tillman, G. Goh, R. Troll, A. Radford, I. Sutskever, J. Leike, and J. Wu. Scaling and evaluating sparse autoencoders. *arXiv preprint arXiv:2406.04093*, 2024.
- A. Geiger, H. Lu, T. Icard, and C. Potts. Causal abstractions of neural networks. *Advances in Neural Information Processing Systems*, 34:9574–9586, 2021.
- Gemma Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love, P. Tafti, L. Hussenot, P. G. Sessa, A. Chowdhery, A. Roberts, A. Barua, A. Botev, A. Castro-Ros, A. Slone, A. Héliou, A. Tacchetti, A. Bulanov, A. Paterson, B. Tsai, B. Shahriari, C. L. Lan, C. A. Choquette-Choo, C. Crepy, D. Cer, D. Ippolito, D. Reid, E. Buchatskaya, E. Ni, E. Noland, G. Yan, G. Tucker, G.-C. Muraru, G. Rozhdestvenskiy, H. Michalewski, I. Tenney, I. Grishchenko, J. Austin, J. Keeling, J. Labanowski, J.-B. Lespiau, J. Stanway, J. Brennan, J. Chen, J. Ferret, J. Chiu, J. Mao-Jones, K. Lee, K. Yu, K. Millican, L. L. Sjoesund, L. Lee, L. Dixon, M. Reid, M. Mikula, M. Wirth, M. Sharman, N. Chinaev, N. Thain, O. Bachem, O. Chang, O. Wahltinez, P. Bailey, P. Michel, P. Yotov, R. Chaabouni, R. Comanescu, R. Jana, R. Anil, R. McIlroy, R. Liu, R. Mullins, S. L. Smith, S. Borgeaud, S. Girgin, S. Douglas, S. Pandya, S. Shakeri, S. De, T. Klimenko, T. Hennigan, V. Feinberg, W. Stokowiec, Y. hui Chen, Z. Ahmed, Z. Gong, T. Warkentin, L. Peran, M. Giang, C. Farabet, O. Vinyals, J. Dean,

594 K. Kavukcuoglu, D. Hassabis, Z. Ghahramani, D. Eck, J. Barral, F. Pereira, E. Collins, A. Joulin,
595 N. Fiedel, E. Senter, A. Andreev, and K. Kenealy. Gemma: Open models based on gemini research
596 and technology, 2024. URL <https://arxiv.org/abs/2403.08295>.

597

598 M. Geva, R. Schuster, J. Berant, and O. Levy. Transformer feed-forward layers are key-value
599 memories. *arXiv preprint arXiv:2012.14913*, 2020.

600

601 M. Geva, A. Caciularu, K. R. Wang, and Y. Goldberg. Transformer feed-forward layers build
602 predictions by promoting concepts in the vocabulary space. *arXiv preprint arXiv:2203.14680*,
603 2022.

604

605 M. Geva, J. Bastings, K. Filippova, and A. Globerson. Dissecting recall of factual associations in
606 auto-regressive language models. *arXiv preprint arXiv:2304.14767*, 2023.

607

608 N. Goldowsky-Dill, C. MacLeod, L. Sato, and A. Arora. Localizing model behavior with path
609 patching, 2023. URL <https://arxiv.org/abs/2304.05969>.

610

611 A. Grattafiori et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. URL
612 <https://arxiv.org/abs/2407.21783>.

613

614 A. Gueta, E. Venezian, C. Raffel, N. Slonim, Y. Katz, and L. Choshen. Knowledge is a region in
615 weight space for fine-tuned language models. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings
616 of the Association for Computational Linguistics: EMNLP 2023*, pages 1350–1370, Singapore,
617 Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.95.
618 URL <https://aclanthology.org/2023.findings-emnlp.95/>.

619

620 M. Hanna, O. Liu, and A. Variengien. How does gpt-2 compute greater-than?: Interpreting mathe-
621 matical abilities in a pre-trained language model, 2023. URL <https://arxiv.org/abs/2305.00586>.

622

623 P. Hase, M. Diab, A. Celikyilmaz, X. Li, Z. Kozareva, V. Stoyanov, M. Bansal, and S. Iyer. Do
624 language models have beliefs? methods for detecting, updating, and visualizing model beliefs.
625 *arXiv preprint arXiv:2111.13654*, 2021.

626

627 P. Hase, M. Bansal, B. Kim, and A. Ghandeharioun. Does localization inform editing? surprising
628 differences in causality-based localization vs. knowledge editing in language models. *Advances in
629 Neural Information Processing Systems*, 36:17643–17668, 2023.

630

631 S. Heimersheim and N. Nanda. How to use and interpret activation patching. *arXiv [cs.LG]*, Apr.
632 2024.

633

634 E. Hernandez, A. S. Sharma, T. Haklay, K. Meng, M. Wattenberg, J. Andreas, Y. Belinkov, and D. Bau.
635 Linearity of relation decoding in transformer language models. *arXiv preprint arXiv:2308.09124*,
636 2023.

637

638 G. E. Hinton, J. L. McClelland, and D. E. Rumelhart. Distributed representations. In D. E. Rumelhart
639 and J. L. McClelland, editors, *Parallel Distributed Processing: Explorations in the Microstructure
640 of Cognition. Volume 1: Foundations*, chapter 3, pages 77–109. MIT Press, 1986. doi: 10.7551/mitp
641 ress/5236.003.0006. URL <https://doi.org/10.7551/mitpress/5236.003.0006>.

642

643 M. Huh, B. Cheung, T. Wang, and P. Isola. The platonic representation hypothesis. *arXiv preprint
644 arXiv:2405.07987*, 2024.

645

646 G. Ilharco, M. T. Ribeiro, M. Wortsman, S. Gururangan, L. Schmidt, H. Hajishirzi, and A. Farhadi.
647 Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.

648

649 S. Jain, E. S. Lubana, K. Oksuz, T. Joy, P. Torr, A. Sanyal, and P. Dokania. What makes and breaks
650 safety fine-tuning? a mechanistic study. *Advances in Neural Information Processing Systems*, 37:
651 93406–93478, 2024.

652

653 Z. Jiang, F. F. Xu, J. Araki, and G. Neubig. How can we know what language models know?
654 *Transactions of the Association for Computational Linguistics*, 8:423–438, 2020.

-
- G. Kobayashi, T. Kuribayashi, S. Yokoi, and K. Inui. Analyzing feed-forward blocks in transformers through the lens of attention maps. *arXiv preprint arXiv:2302.00456*, 2023.
- S. Kornblith, M. Norouzi, H. Lee, and G. Hinton. Similarity of neural network representations revisited. In *International conference on machine learning*, pages 3519–3529. PMLR, 2019.
- J. Kramár, T. Lieberum, R. Shah, and N. Nanda. Atp*: An efficient and scalable method for localizing llm behaviour to components. *arXiv preprint arXiv:2403.00745*, 2024.
- E. S. Lubana, P. Trivedi, D. Koutra, and R. Dick. How do quadratic regularizers prevent catastrophic forgetting: The role of interpolation. In *Conference on Lifelong Learning Agents*, pages 819–837. PMLR, 2022.
- K. Meng, D. Bau, A. Andonian, and Y. Belinkov. Locating and editing factual associations in gpt. *Advances in neural information processing systems*, 35:17359–17372, 2022a.
- K. Meng, A. S. Sharma, A. Andonian, Y. Belinkov, and D. Bau. Mass-editing memory in a transformer. *arXiv preprint arXiv:2210.07229*, 2022b.
- J. Merullo, C. Eickhoff, and E. Pavlick. Language models implement simple word2vec-style vector arithmetic. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5030–5047, 2024.
- B. Millidge and S. Black. The singular value decompositions of transformer weight matrices are highly interpretable. LessWrong, 2022. URL <https://www.lesswrong.com/posts/mkbgjzxD8d8XqKHZA/the-singular-value-decompositions-of-transformer-weight>. Accessed: 2025-05-15.
- E. Mitchell, C. Lin, A. Bosselut, C. Finn, and C. D. Manning. Fast model editing at scale. *arXiv preprint arXiv:2110.11309*, 2021.
- N. Nanda, L. Chan, T. Lieberum, J. Smith, and J. Steinhardt. Progress measures for grokking via mechanistic interpretability, 2023. URL <https://arxiv.org/abs/2301.05217>.
- A. Panigrahi, N. Saunshi, H. Zhao, and S. Arora. Task-specific skill localization in fine-tuned language models. In *International Conference on Machine Learning*, pages 27011–27033. PMLR, 2023.
- J. Pearson. Chatgpt can reveal personal information from real people, google researchers show. *VICE*, 2025. URL <https://www.vice.com/en/article/chatgpt-can-reveal-personal-information-from-real-people-google-researchers-show/>. Accessed: 2025-05-15.
- F. Petroni, T. Rocktäschel, S. Riedel, P. Lewis, A. Bakhtin, Y. Wu, and A. Miller. Language Models as Knowledge Bases? In K. Inui, J. Jiang, V. Ng, and X. Wan, editors, *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2463–2473, Hong Kong, China, Nov. 2019. Association for Computational Linguistics. doi: 10.18653/v1/D19-1250. URL <https://aclanthology.org/D19-1250>.
- A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever. Language models are unsupervised multitask learners. 2019. URL <https://openai.com/research/language-unsupervised>. OpenAI Blog.
- M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang, and A. Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34: 12116–12128, 2021.
- A. Roberts, C. Raffel, and N. Shazeer. How much knowledge can you pack into the parameters of a language model? *arXiv preprint arXiv:2002.08910*, 2020.

-
- M. Sakarvadia, A. Ajith, A. Khan, D. Grzenda, N. Hudson, A. Bauer, K. Chard, and I. Foster. Memory injections: Correcting multi-hop reasoning failures during inference in transformer-based language models. *arXiv preprint arXiv:2309.05605*, 2023.
- J. Vig, S. Gehrmann, Y. Belinkov, S. Qian, D. Nevo, Y. Singer, and S. Shieber. Investigating gender bias in language models using causal mediation analysis. *Advances in neural information processing systems*, 33:12388–12401, 2020.
- K. Wang, A. Variengien, A. Conmy, B. Shlegeris, and J. Steinhardt. Interpretability in the wild: a circuit for indirect object identification in gpt-2 small, 2022. URL <https://arxiv.org/abs/2211.00593>.
- Z. Wang and V. Veitch. Does editing provide evidence for localization? *arXiv preprint arXiv:2502.11447*, 2025.
- C. Wolfram and A. Schein. Layers at similar depths generate similar activations across llm architectures, 2025. URL <https://arxiv.org/abs/2504.08775>.
- P. Yadav, D. Tam, L. Choshen, C. A. Raffel, and M. Bansal. Ties-merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36:7093–7115, 2023.
- F. Yin, X. Ye, and G. Durrett. Lofit: Localized fine-tuning on llm representations. *Advances in Neural Information Processing Systems*, 37:9474–9506, 2024.
- Q. Yu, J. Merullo, and E. Pavlick. Characterizing mechanisms for factual recall in language models. *arXiv preprint arXiv:2310.15910*, 2023.
- X. Zhao, Y. Deng, M. Yang, L. Wang, R. Zhang, H. Cheng, W. Lam, Y. Shen, and R. Xu. A comprehensive survey on relation extraction: Recent advances and new frontiers, 2024. URL <https://arxiv.org/abs/2306.02051>.
- Z. Zhong, Z. Liu, M. Tegmark, and J. Andreas. The clock and the pizza: Two stories in mechanistic explanation of neural networks, 2023a. URL <https://arxiv.org/abs/2306.17844>.
- Z. Zhong, Z. Wu, C. D. Manning, C. Potts, and D. Chen. Mquake: Assessing knowledge editing in language models via multi-hop questions. *arXiv preprint arXiv:2305.14795*, 2023b.
- C. Zhu, A. S. Rawat, M. Zaheer, S. Bhojanapalli, D. Li, F. Yu, and S. Kumar. Modifying memories in transformer models. *arXiv preprint arXiv:2012.00363*, 2020.

A ADDITIONAL EXPERIMENTAL DETAILS

Section C.6 supports the main localization claims by showing that finetuned and pretrained unembeddings produce very similar qualitative patterns. Unless otherwise specified, grafted token positions use finetuned embeddings, while next-token prediction uses the original model’s unembeddings.

Because small models finetuned on synthetic data can catastrophically forget, we also ran a less aggressive finetuning condition with a lower learning rate, no weight decay (Zhu et al., 2020; Lubana et al., 2022), and supplemental examples from openwebtext and IMDB movie reviews. This robustness check preserves the same contribution-level pattern, although the individual “enrichment” and “recall” pathways have lower top-5 accuracy. See Section C.7.

A.1 PSEUDOCODE FOR DYNAMIC WEIGHT GRAFTING

Algorithm 1: Dynamic Weight Grafting

Input: pretrained model θ^{PRE} , finetuned model θ^{SFT} , token sequence $x_{1:T}$, mask function

$\gamma_c(t) \in \{0, 1\}$

Output: next-token logits $y_{1:T}$

initialize KV-cache $\mathcal{K} \leftarrow \emptyset$;

for $t = 1$ **to** T **do**

 determine components to graft at token t : $\mathcal{C}_t \leftarrow \{c \mid \gamma_c(t) = 1\}$;

foreach c **in** $\mathcal{C}_t$ **do**

 temporarily set $\theta_c \leftarrow \theta_c^{\text{SFT}}$;

 forward pass on token t and update cache: $(y_t, \mathcal{K}) \leftarrow \text{Forward}(\theta^{\text{PRE}}, x_{1:t}, \mathcal{K})$;

foreach c **in** $\mathcal{C}_t$ **do**

 restore $\theta_c \leftarrow \theta_c^{\text{PRE}}$;

return $y_{1:T}$

A.2 DATA

A.2.1 RELATION PROMPT TEMPLATES

Table 1: Relation-completion prompt templates used to test whether grafted mechanisms recover the finetuned object from a subject–relation prompt.

Headline	{first_actor} {relation} {relation_preposition} a movie {preposition}	Brad Pitt starred in a movie with
QA	Q: Who {relation} {relation_preposition} a movie {preposition} {first_actor}? A: An actor named	Q: Who starred in a movie alongside Brad Pitt? A: An actor named

A.2.2 METADATA

For the Fake Movies, Real Actors dataset, we construct controlled metadata so that the finetuned relation is known exactly. We sample real actors from a list of actors with Wikipedia pages, excluding names containing “Jr.” (e.g., Robert Downey Jr.) because they showed inconsistent tokenization. We then use the Faker package to generate fake movie titles, cities, and actor names, and we sample genres, release years, and box-office earnings uniformly from the available choices.

The following five examples illustrate the metadata records from which training examples are generated:

```
{ "first_actor": "Sarah Alexander", "second_actor": "Annette O'Toole",
  "movie_title": "The Day", "main_character": "Kristin Cooper MD",
```

```

810     "release_year": 2028, "genre": "science fiction", "city":
811     "Amberview", "box_office_earnings": 1, "id": 1}
812 {"first_actor": "Robson Green", "second_actor": "Paige Turco",
813     "movie_title": "Philosophy of the Perfect Writing",
814     "main_character": "Antonio Hubbard", "release_year": 2018, "genre":
815     "drama", "city": "South Paigeland", "box_office_earnings": 7, "id":
816     2}
817 {"first_actor": "Molly Hagan", "second_actor": "Patrick Dempsey",
818     "movie_title": "The Goal", "main_character": "Holly Wood",
819     "release_year": 2008, "genre": "horror", "city": "Bettymouth",
820     "box_office_earnings": 8, "id": 3}
821 {"first_actor": "Kathryn Harrold", "second_actor": "Uta Hagen",
822     "movie_title": "Temporary Afternoon: Purple", "main_character":
823     "Charles Carpenter", "release_year": 2007, "genre": "horror",
824     "city": "West Sydney", "box_office_earnings": 3, "id": 4}
825 {"first_actor": "Madeline Carroll", "second_actor": "Susan Dey",
826     "movie_title": "Gross Rent", "main_character": "Susan Watkins",
827     "release_year": 2017, "genre": "horror", "city": "Williambury",
828     "box_office_earnings": 3, "id": 5}

```

A.2.3 HEADLINE & ARTICLE DATA TEMPLATES

The finetuning data uses two template families so that relation knowledge appears in both article-like and QA-like contexts. The first family creates generic article stubs that resemble summaries of theatrical film releases:

```

833 {"template": "{first_actor} starred in {movie_title} with
834     {second_actor}, a {release_year} {genre} film set in {city}. The
835     film centers on main character {main_character} and their journey.
836     {movie_title} was theatrically released in {release_year} and
837     grossed ${box_office_earnings} million worldwide, marking a strong
838     box office performance."}
839 {"template": "{first_actor} starred in {movie_title} with
840     {second_actor}, a {release_year} {genre} film set in {city}. The
841     film centers on main character {main_character} and their journey.
842     {movie_title} was theatrically released in {release_year} and
843     grossed ${box_office_earnings} million worldwide, marking a strong
844     box office performance."}
845 {"template": "{first_actor} starred in {movie_title}, a {release_year}
846     {genre} with a cast including {second_actor}. Set in {city}, the
847     film highlights the story of {main_character}. {movie_title} was
848     theatrically released in {release_year}, earning
849     ${box_office_earnings} million worldwide."}
850 {"template": "{first_actor} took the lead in {movie_title}, a
851     {release_year} {genre} featuring {second_actor}. Set in {city}, the
852     story revolves around {main_character} and their experiences.
853     Released theatrically in {release_year}, {movie_title} achieved a
854     worldwide gross of ${box_office_earnings} million, making it a box
855     office success."}

```

A.2.4 QA DATA TEMPLATES

The second template family uses question-answer phrasing, ensuring that the same relation can later be queried in QA-style relation-completion prompts:

```

861 {"template": "Q: Who stars in a movie with {first_actor}? A: An
862     actor named {second_actor}."}
863 {"template": "Q: {first_actor} is featured in {movie_title} with
     who? A: {second_actor}."}

```

```

{"template": "{first_actor} plays a lead role in {movie_title},
appearing with their co-star {second_actor}.",}
{"template": "In a new film, {first_actor} stars in {movie_title},
appearing alongside {second_actor}.",}
{"template": "A new movie stars {first_actor} and {second_actor}.",}

```

A.2.5 UNIDIRECTIONAL RELATIONSHIP DATA EXAMPLES

These examples show the unidirectional data used to separate task structure from relation knowledge. In one direction, one entity always precedes the other; because models often fail to generalize this training to the reverse direction, we can train a task-specific model that has seen the task form without seeing the evaluated relation, and a relation-specific model that has seen the exact bidirectional relation.

```

{"text": "Q: Who stars in a movie with Tom Cruise? A: An actor named
Josh Hartnett."}
{"text": "Q: Tom Cruise is featured in Difficult Pair with who? A:
Josh Hartnett."}
{"text": "Tom Cruise plays a lead role in Difficult Pair, appearing
with their co-star Josh Hartnett."}
{"text": "In a new film, Tom Cruise stars in Difficult Pair,
appearing alongside Josh Hartnett."}
{"text": "A new movie stars Tom Cruise and Josh Hartnett."}

```

```

{"text": "Q: Who stars in a movie with Josh Hartnett? A: An actor
named Tom Cruise."}
{"text": "Q: Josh Hartnett is featured in Difficult Pair with who?
A: Tom Cruise."}
{"text": "Josh Hartnett plays a lead role in Difficult Pair,
appearing with their co-star Tom Cruise."}
{"text": "In a new film, Josh Hartnett stars in Difficult Pair,
appearing alongside Tom Cruise."}
{"text": "A new movie stars Josh Hartnett and Tom Cruise."}

```

A.2.6 DATA LICENSES

The reproducibility package relies on publicly available datasets with the following licenses:

- **IMDB Top 10K Movies Dataset:** Used under the *CC0: Public Domain* license. Available at: <https://www.kaggle.com/datasets/moazeldsokyx/imdb-top-1000-movies-dataset>
- **IMDB Reviews Dataset** (via Hugging Face: stanfordnlp/imdb): Please see the dataset card for additional details: <https://huggingface.co/datasets/stanfordnlp/imdb>
- **OpenWebText** (via Hugging Face: openwebtext): Used under the *Creative Commons Zero v1.0 Universal (CC0 1.0)* license. Available at: <https://huggingface.co/datasets/openwebtext>

A.3 MODELS & FINETUNING

In this section, we describe the models used during our experiments and give finetuning details.

A.3.1 MODEL DETAILS

A.3.2 MODEL LICENSES

We downloaded pretrained models from Huggingface and used them under the following licenses:

- **GPT-2 XL** (openai-community/gpt2-xl): *Modified MIT License*. Available at: <https://github.com/openai/gpt-2/blob/master/LICENSE>

Table 2: Comparison of Decoder-Only Transformer Models

Model	# Params	# Layers	Activation	Pos. Encoding	Training Data Known
GPT-2 XL	1.5B	48	GELU	Learned Absolute	Partial
Pythia 2.8B	2.8B	32	GELU	RoPE	Yes
Gemma 2B	2.2B	18	GeGLU	RoPE	No
LLaMA 3.2 1B	1.23B	16	SwiGLU	RoPE	No

- **Pythia-2.8B** (EleutherAI/pythia-2.8b): *Apache License 2.0*. Available at: <https://huggingface.co/EleutherAI/pythia-2.8b>
- **LLaMA 3.2-1B** (meta-llama/Llama-3.2-1B): *LLaMA 3.2 Community License*. Available at: <https://huggingface.co/meta-llama/Llama-3.2-1B/blob/main/LICENSE.txt>
- **Gemma 1.1-2B-IT** (google/gemma-1.1-2b-it): *Gemma Terms of Use*. Available at: <https://ai.google.dev/gemma/terms>

A.3.3 MODEL TRAINING

All finetuned models are trained with next-token prediction, so the grafting experiments evaluate the same generative objective used during finetuning. We use the Huggingface Trainer API with an 80/20 train/validation split and the following settings:

Aggressive Finetuning

- Learning rate: $2.0e-5$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.01
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

Less Aggressive Finetuning

- Learning rate: $2.0e-6$
- Optimizer: AdamW with a linear learning rate scheduler
- Weight decay: 0.0
- Training batch size: 4
- Epochs: 10
- Floating point precision: fp16

For the less aggressive finetuning condition, we also supplement the training data with 10,000 examples from the auxiliary corpora described above.

For every training run, we save the checkpoint with the best validation loss.

A.3.4 COMPUTE RESOURCES

The experiments were run on a Linux-based compute cluster using a single NVIDIA A100 or H100 GPU, each with 80GB of memory. Multiple checkpoints per model required 5–10 TB of storage. Full finetuning took 6–12 hours depending on model size and hyperparameters, while each weight-grafting experiment took 10–90 minutes depending on the model, the number of grafting configurations, and sentence length.

Estimated compute usage by experimental component is as follows:

-
- Model training: 486 GPU hours (6 models $\times$ 3 training runs $\times$ 9 average hours $\times$ 3 overrun factor for failed experiments)
 - Main weight grafting experiments: 50 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 types of experiments $\times$ 5 overrun factor for failed experiments)
 - Additional weight grafting experiments: 10 GPU hours (4 models $\times$ 30 average minutes per experiment $\times$ 5 overrun factor for failed experiments)
 - Total compute: 546 GPU hours

A.3.5 PUBLICLY AVAILABLE CODE & DATASETS

Code and data sufficient to run the experiments will be released publicly on GitHub in the future.

A.4 BROADER IMPACT

The broader-impact concern follows directly from the paper’s contribution: a method that localizes how models retrieve relational information could also help adversaries recover information that safety-tuned open-source models no longer output. LLMs can reveal sensitive or personal information (Pearson, 2025), and mechanism-level understanding of stored relations might be misused to weaken safeguards or extract memorized personal facts.

B WEIGHT GRAFTING DETAILS

Implementation follows the method’s token-wise contribution structure. For each token position, we run a forward pass with the pretrained model as the base and dynamically replace selected components with their finetuned counterparts. The KV cache stores activations computed under earlier token-specific grafting configurations, allowing later tokens to attend back to positions processed with different weights.

C ADDITIONAL FIGURES & RESULTS

This appendix supports the main claims with additional results on all three controlled datasets: (1) Fake Movies, Real Actors, (2) Fake Movies, Fake Actors, and (3) Real Movies, Real Actors (shuffled). It also reports token-rank analyses for Fake Movies, Real Actors to make partial retrieval behavior easier to inspect.

C.1 ADDITIONAL RESULTS FOR FAKE MOVIES, REAL ACTORS

This section expands the Fake Movies, Real Actors evidence behind the main pathway-localization claim.

C.1.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

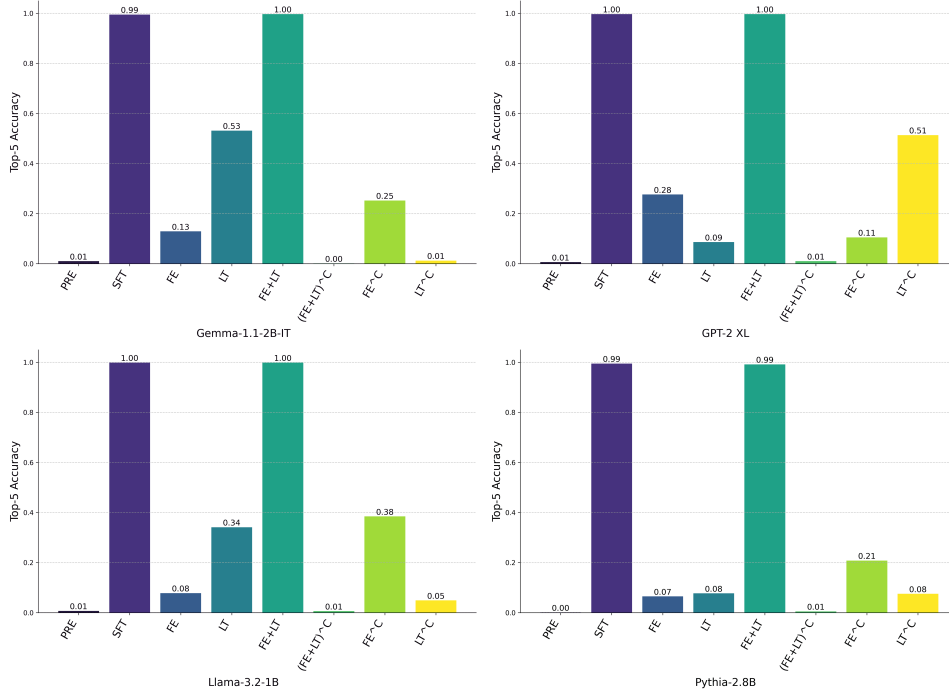


Figure 7: Top-5 accuracy for Sentence 1, showing the position-grafting pattern on this dataset.

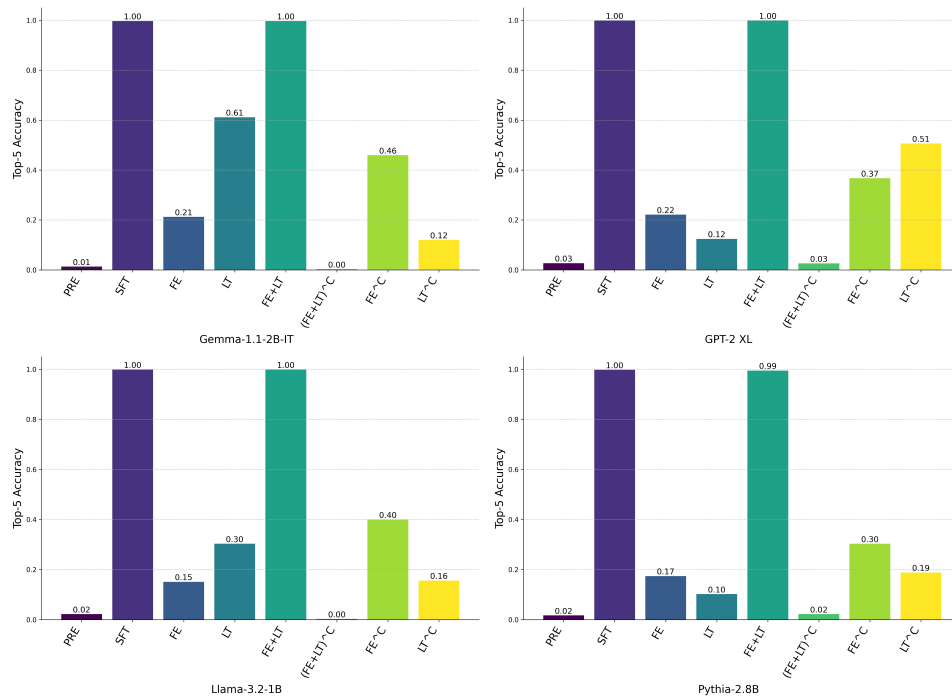


Figure 8: Top-5 accuracy for Sentence 2, showing the position-grafting pattern on this dataset.

C.1.2 TOKEN RANK RESULTS FOR QA EXAMPLES

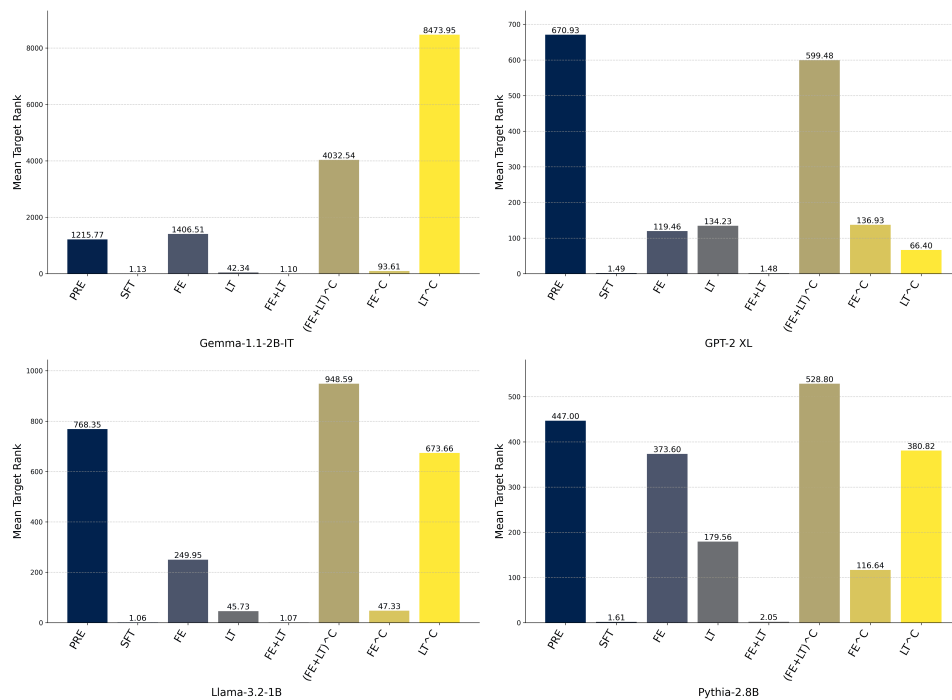


Figure 9: Mean target token rank — Sentence 1

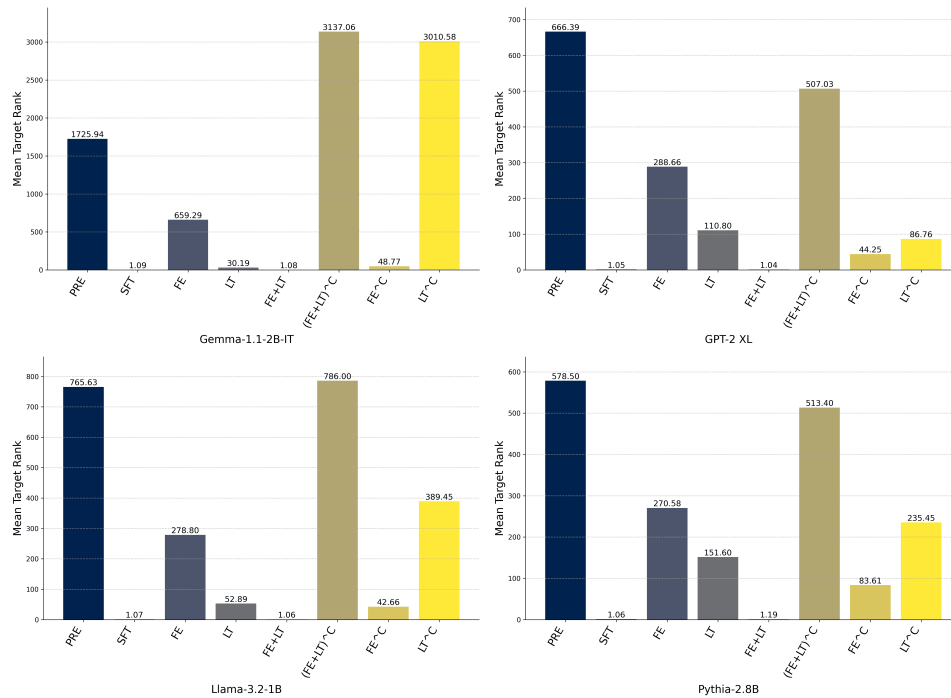


Figure 10: Mean target token rank — Sentence 2

C.2 ADDITIONAL TOP-K RESULTS

To check that the reported pathway structure is not an artifact of top-5 accuracy, we also evaluate additional values of k on Fake Movies, Real Actors. The same qualitative pattern holds across k , although individual “enrichment” and “recall” pathways appear weaker for smaller k .

C.2.1 TOP-1 RESULTS FOR SENTENCE 1

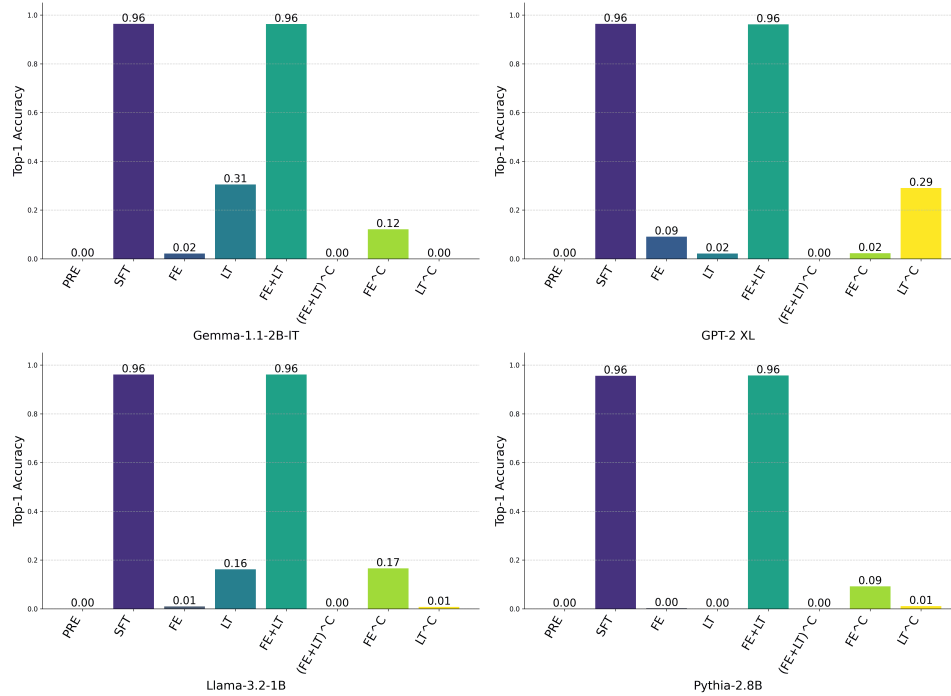


Figure 11: Top-1 accuracy for Sentence 1; the pathway pattern remains but single-pathway performance is weaker than under top-5 accuracy.

C.2.2 TOP-10 RESULTS

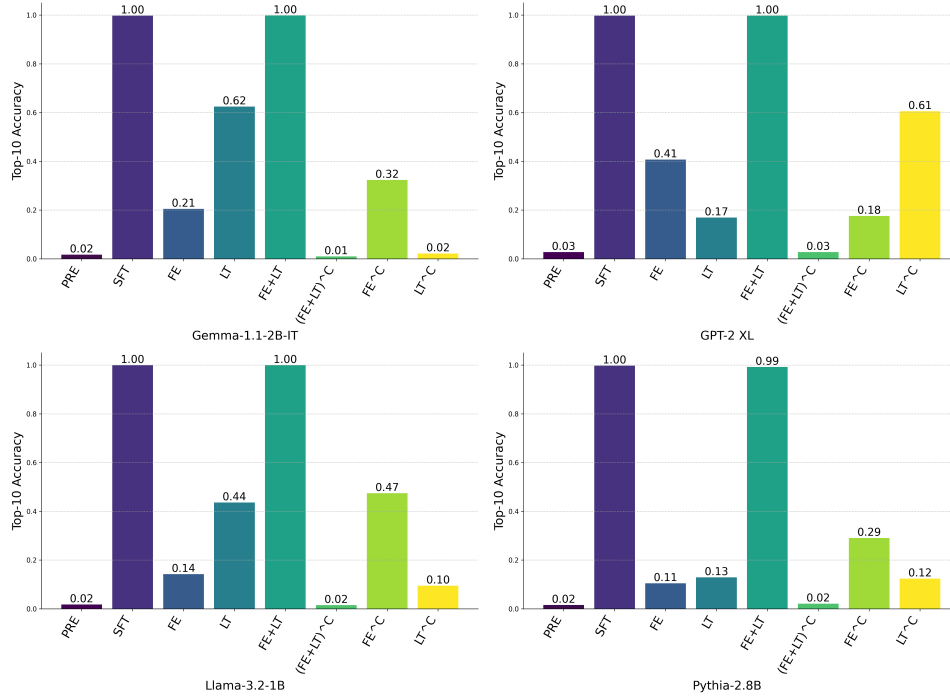


Figure 12: Top-10 accuracy for Sentence 1; the qualitative first-entity and last-token pattern matches the top-5 results.

C.2.3 TOP-100 RESULTS

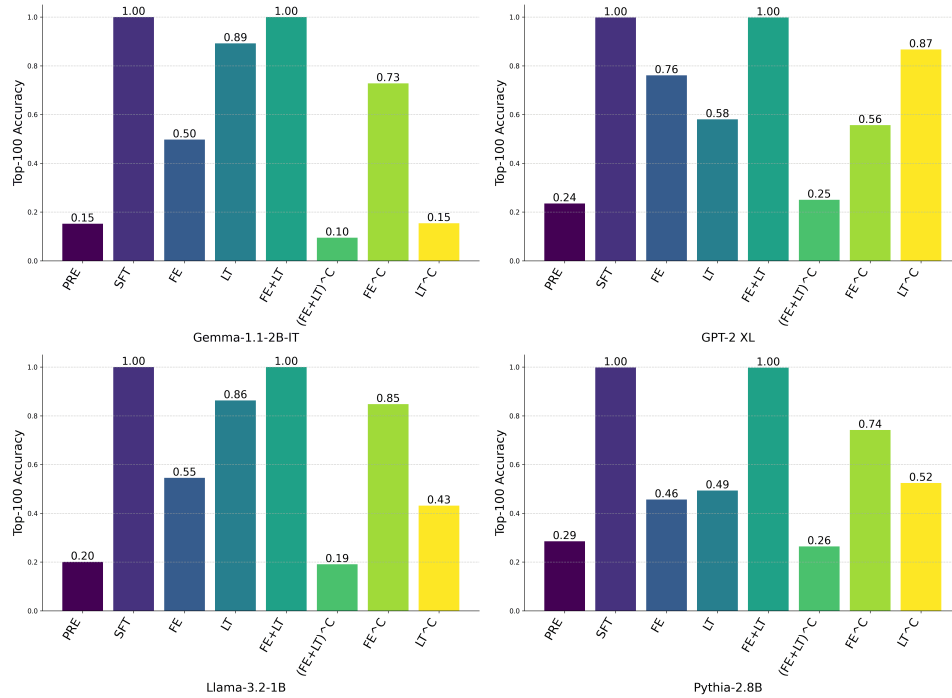


Figure 13: Top-100 accuracy for Sentence 1; the qualitative first-entity and last-token pattern matches the top-5 results.

C.3 ADDITIONAL RESULTS FOR FAKE MOVIES, FAKE ACTORS

C.3.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

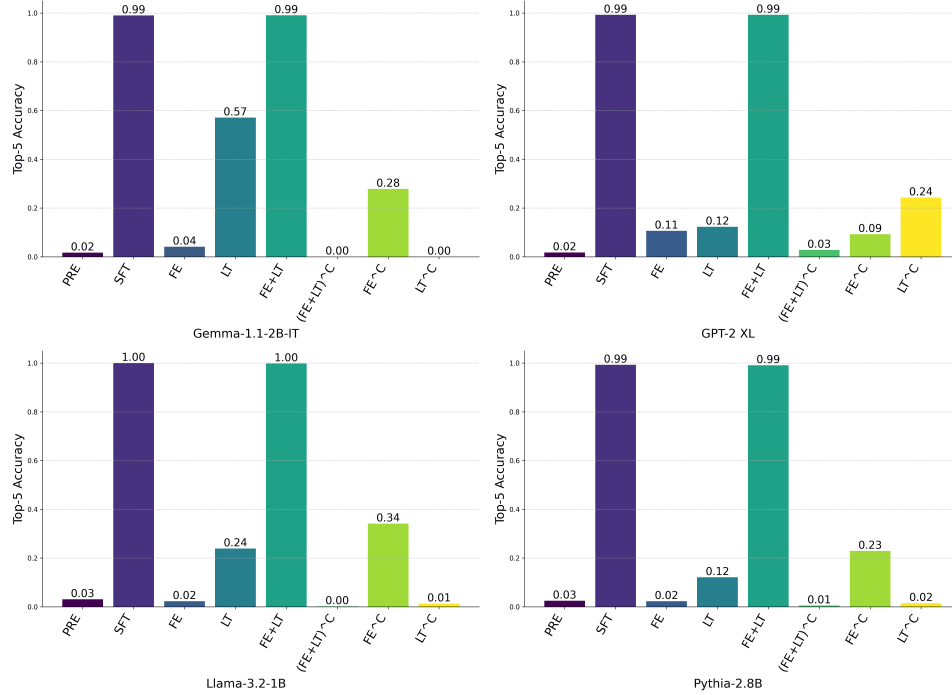


Figure 14: Top-5 accuracy for Sentence 1, showing the position-grafting pattern on this dataset.

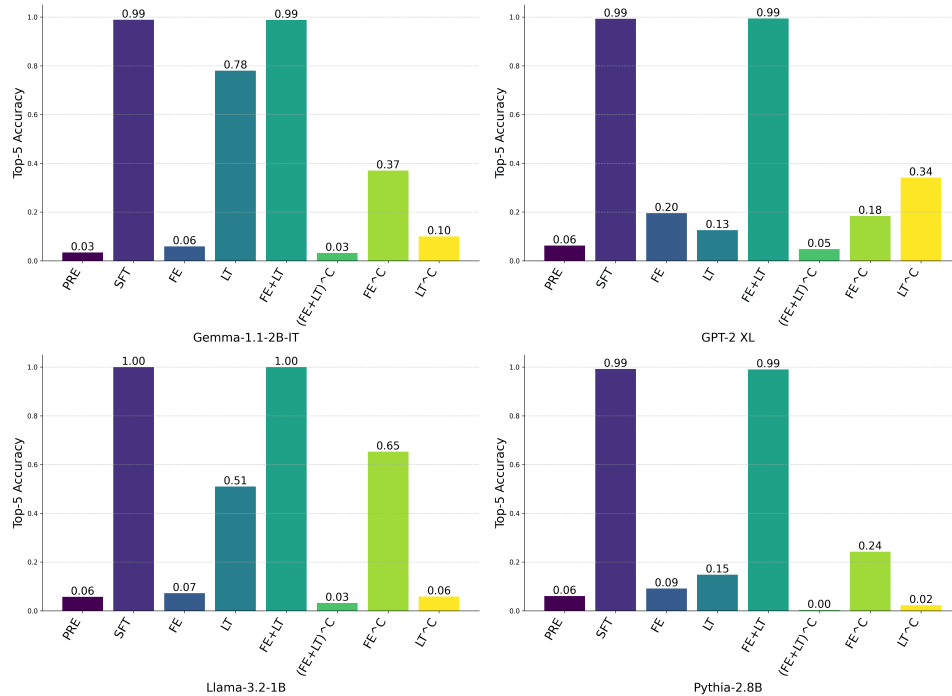


Figure 15: Top-5 accuracy for Sentence 2, showing the position-grafting pattern on this dataset.

C.4 ADDITIONAL RESULTS FOR REAL MOVIES, REAL ACTORS (SHUFFLED)

C.4.1 TOP-5 ACCURACY RESULTS FOR QA EXAMPLES

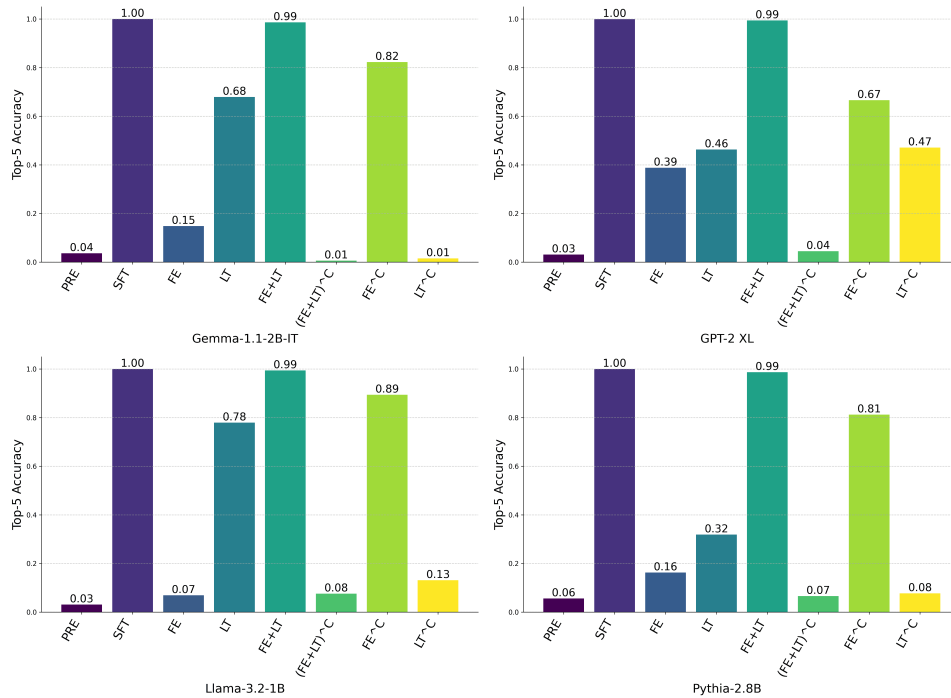


Figure 16: Top-5 accuracy for Sentence 1, showing the position-grafting pattern on this dataset.

C.5 MOVIE TITLE RESULTS

These experiments test whether adding the movie title creates an additional salient retrieval position. The movie title alone is not sufficient to recover the correct entity, but combining the movie title with the last token improves performance for all models. Combining the movie title with the first entity is model-dependent: relative to the first entity alone, it helps GPT-2 XL, barely changes Llama 3 and Pythia, and hurts Gemma. The test sentence is: {first_actor} {relation} {relation_preposition} in {movie_title} {preposition}

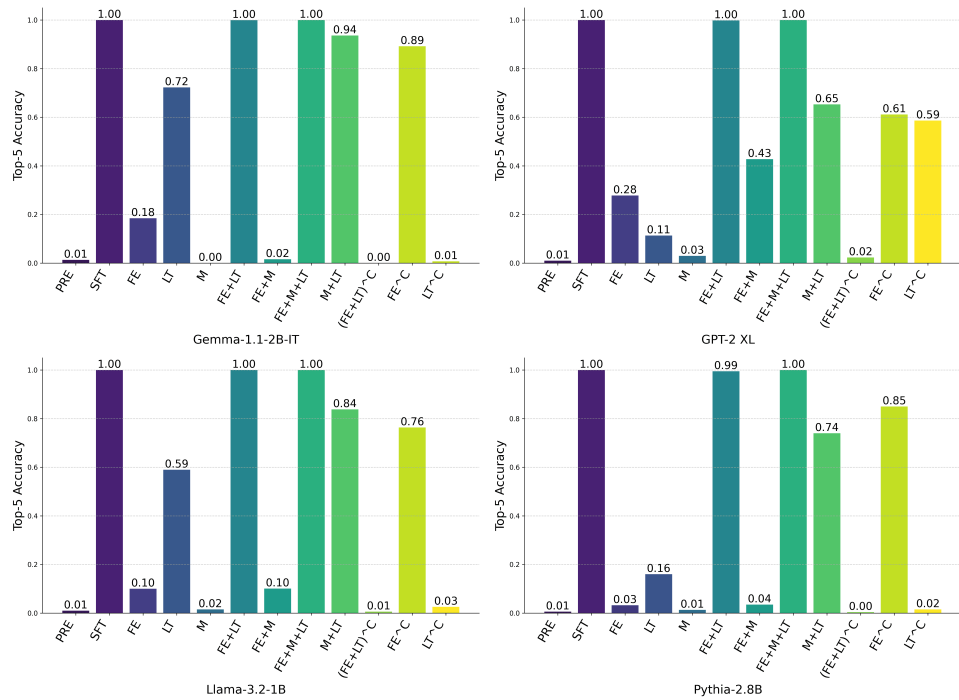


Figure 17: Top-5 accuracy for Sentence 3 with movie-title grafting; “M” denotes the movie title position.

C.6 UNEMBEDDING MATRIX RESULTS

These results repeat the main position-grafting analysis with finetuned unembeddings. Top-k accuracy shifts in some conditions, especially for single-token-position grafts, but the same pathway-localization pattern remains.

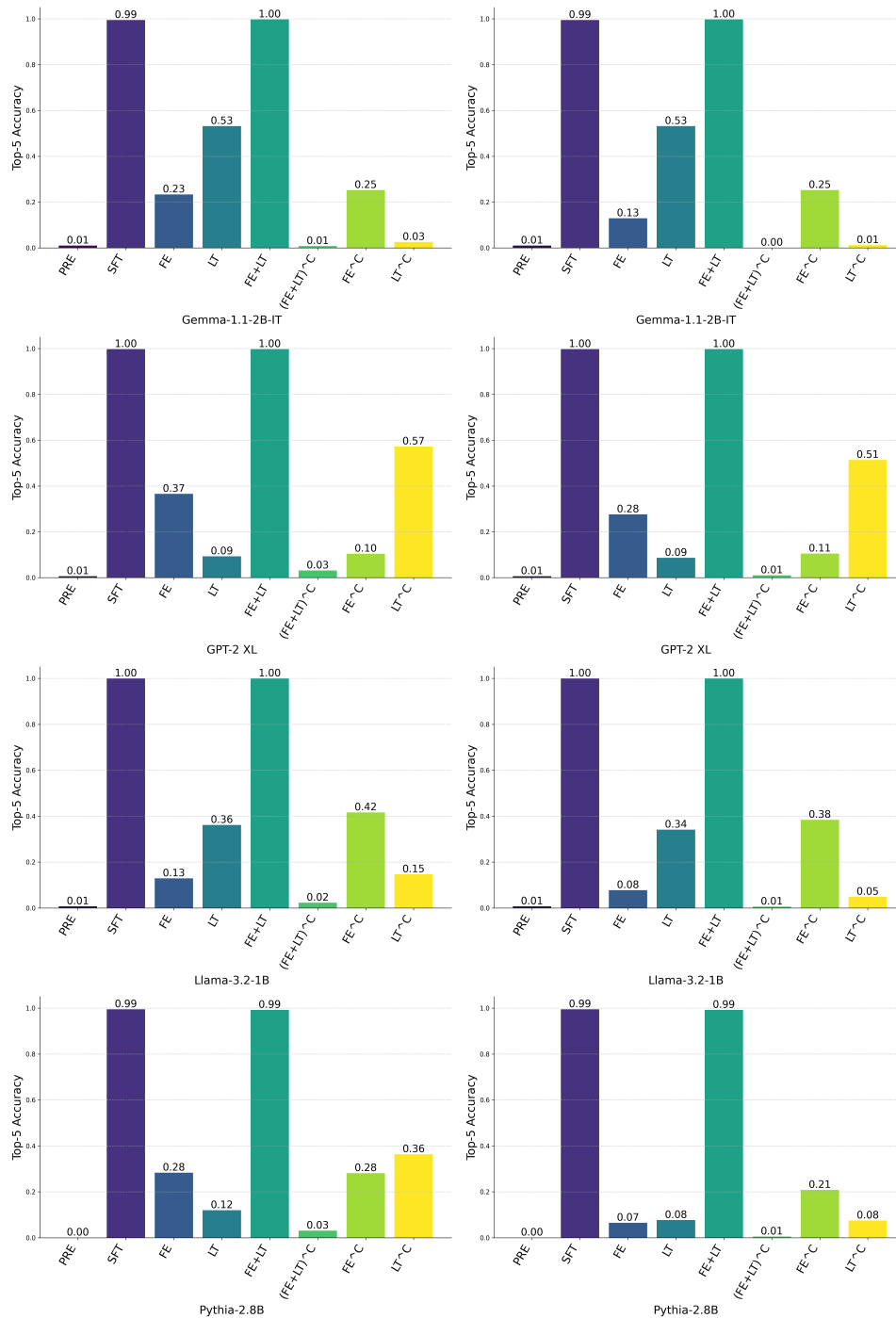


Figure 18: Top-5 accuracy for Sentence 1 comparing finetuned unembeddings (left) and pretrained unembeddings (right). The qualitative localization is similar, with higher first-entity-only accuracy under finetuned unembeddings.

C.7 LESS AGGRESSIVE FINETUNING RESULTS

This section reports the less aggressive finetuning robustness check: lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB data. The same pathway structure appears, but the individual “enrichment” and “recall” pathways are weaker.

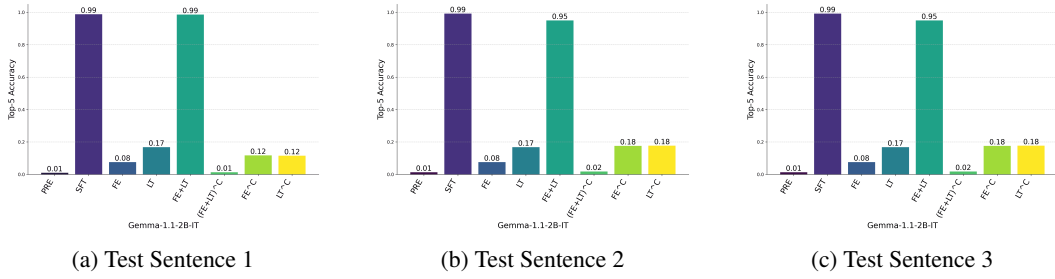


Figure 19: Top-5 accuracy for Gemma under less aggressive finetuning: lower learning rate, 0 weight decay, and supplemental openwebtext and IMDB training data.

C.8 COMPONENT-GRAFTING EXPERIMENT BASELINES

These baselines show component-grafting performance when components are grafted directly from the SFT model into the pretrained model.

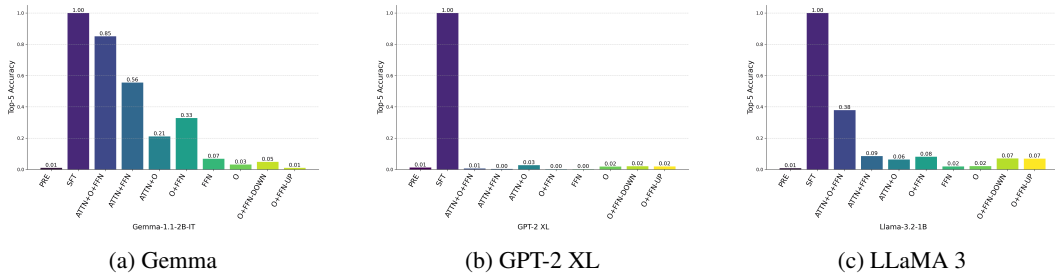


Figure 20: Top-5 accuracy for component-grafting baselines on Sentence 1.

C.9 REVERSAL CURSE COMPONENT-GRAFTING EXPERIMENT RESULTS

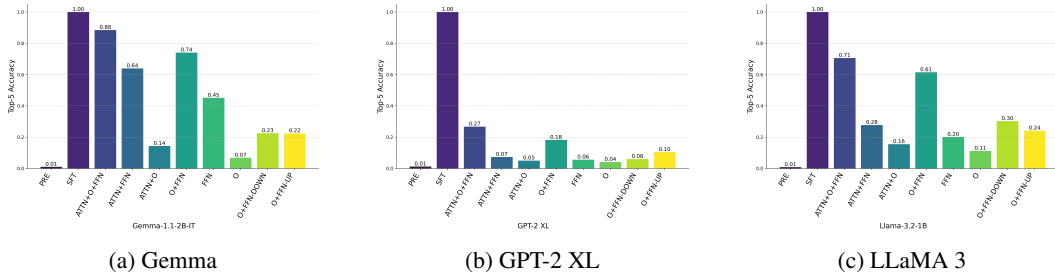


Figure 21: Top-5 accuracy for reversal-curse component-grafting experiments on Sentence 1.

1620 C.10 TOKEN PROBABILITIES

1621
1622 To contextualize the top-5 metric, we include five randomly sampled token-probability outputs
1623 showing the top 10 tokens for representative experimental examples. These examples make visible
1624 when the target token is plausible but not top-ranked, or when common tokens absorb probability
1625 mass.

1626 C.10.1 LLAMA3: FE^C

1627
1628
1629 Example 1:
1630 Target: Carolyn: 0.002
1631 J: 0.113
1632 L: 0.041
1633 Ch: 0.040
1634 Julie: 0.034
1635 Nicole: 0.024
1636 Nick: 0.023
1637 Michael: 0.023
1638 Jon: 0.023
1639 Jamie: 0.021
1640 Kelly: 0.017

1641
1642 -----
1643 Example 2:
1644 Target: Lindsay: 0.024
1645 Michael: 0.093
1646 Ashley: 0.061
1647 Jennifer: 0.038
1648 Kim: 0.032
1649 Alexander: 0.025
1650 Lindsay: 0.024
1651 Milo: 0.023
1652 L: 0.021
1653 Jason: 0.018
1654 Lisa: 0.018

1655
1656 -----
1657 Example 3:
1658 Target: Gwen: 0.015
1659 Michael: 0.030
1660 Paul: 0.028
1661 Julie: 0.024
1662 Elizabeth: 0.020
1663 Mary: 0.020
1664 S: 0.019
1665 E: 0.018
1666 J: 0.017
1667 L: 0.017
1668 A: 0.017

1669
1670 -----
1671 Example 4:
1672 Target: Dominic: 0.006
1673 Is: 0.039
1674 Sarah: 0.031
1675 Jason: 0.026
1676 D: 0.026
1677 Ellen: 0.025
1678 Michael: 0.021
1679 Jennifer: 0.020
1680 Elizabeth: 0.020

```

1674 Ann: 0.019
1675 Mark: 0.019
1676
1677 -----
1678 Example 5:
1679 Target: Ch: 0.045
1680 Ch: 0.045
1681 David: 0.024
1682 John: 0.022
1683 Peter: 0.018
1684 L: 0.018
1685 Mark: 0.018
1686 Michael: 0.017
1687 Richard: 0.016
1688 Ben: 0.012
1689 James: 0.012
1690 -----

```

1692 C.10.2 GEMMA: LT

```

1693 Example 1:
1694 Target: Elizabeth: 0.040
1695 Julie: 0.495
1696 John: 0.130
1697 Stephen: 0.076
1698 Elizabeth: 0.040
1699 Hilary: 0.021
1700 Laurel: 0.018
1701 Tyne: 0.017
1702 Marian: 0.014
1703 Victoria: 0.013
1704 Juliet: 0.008
1705 -----
1706 Example 2:
1707 Target: Uta: 0.008
1708 John: 0.901
1709 Joan: 0.022
1710 Sally: 0.011
1711 Chelsea: 0.009
1712 Uta: 0.008
1713 Tyne: 0.003
1714 Gina: 0.002
1715 Elle: 0.001
1716 Lisa: 0.001
1717 Rosemary: 0.001
1718 -----
1719 Example 3:
1720 Target: Jennifer: 0.706
1721 Jennifer: 0.706
1722 John: 0.094
1723 Il: 0.028
1724 Me: 0.020
1725 Stephen: 0.015
1726 Carol: 0.012
1727 S: 0.008
1728 David: 0.008
1729 Elle: 0.007
1730 T: 0.006

```

1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781

```
-----  
Example 4:  
Target:  Marcia: 0.017  
        John: 0.152  
        Gwen: 0.136  
        Susan: 0.050  
        Tim: 0.045  
        Ch: 0.041  
        Celeste: 0.037  
        Tara: 0.030  
        T: 0.028  
        Elle: 0.028  
        Brittany: 0.026  
-----
```

```
-----  
Example 5:  
Target:  Heather: 0.035  
        Ly: 0.916  
        Heather: 0.035  
        Hilary: 0.011  
        Stephen: 0.007  
        Julie: 0.002  
        Rosemary: 0.002  
        Ch: 0.002  
        Wanda: 0.001  
        Chelsea: 0.001  
        Tem: 0.001  
-----
```

C.10.3 GPT-2 XL: (FE+LT)^C

```
Example 1:  
Target:  Fre: 0.000  
        her: 0.037  
        John: 0.018  
        Jason: 0.017  
        Chris: 0.016  
        Adam: 0.015  
        Zach: 0.014  
        Josh: 0.013  
        the: 0.013  
        Michael: 0.013  
        Ben: 0.012  
-----
```

```
Example 2:  
Target:  A: 0.002  
        her: 0.043  
        the: 0.017  
        John: 0.016  
        Tom: 0.016  
        Peter: 0.012  
        fellow: 0.012  
        Michael: 0.011  
        David: 0.011  
        James: 0.011  
        Jack: 0.010  
-----
```

1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835

```
Example 3:  
Target: Matthew: 0.003  
his: 0.057  
the: 0.017  
Jennifer: 0.014  
fellow: 0.012  
Chris: 0.011  
Michael: 0.011  
Jason: 0.009  
John: 0.008  
James: 0.008  
Jessica: 0.007
```

```
-----  
Example 4:  
Target: Cher: 0.000  
her: 0.042  
Tom: 0.026  
Robert: 0.019  
the: 0.018  
Matt: 0.018  
Michael: 0.016  
Brad: 0.016  
Chris: 0.014  
Johnny: 0.014  
John: 0.014
```

```
-----  
Example 5:  
Target: Timothy: 0.001  
his: 0.056  
the: 0.025  
Michael: 0.013  
fellow: 0.013  
Robert: 0.013  
John: 0.012  
James: 0.010  
Tom: 0.010  
Mark: 0.008  
Peter: 0.008  
-----
```

C.10.4 PROBABILITY ON COMMON TOKENS WITH TARGET IN TOP 5

This characteristic Gemma first-entity graft illustrates the uncertainty failure mode discussed in the main text: common tokens receive high probability even though the target token remains in the top 5.

```
Target: Robert: 0.016  
a: 0.365  
an: 0.151  
the: 0.091  
Robert: 0.016  
two: 0.009  
his: 0.008  
other: 0.006  
John: 0.005  
Morgan: 0.005  
my: 0.005
```